UNIT V – VIRTUAL FUNCTIONS, POLYMORPHISM & EXCEPTION HANDLING

Name: Moshitha K
Register Number: 192421205
Subject: Object-Oriented Programming Using C++

SECTION A – DEBUGGING QUESTIONS

QUESTION 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

1. Why is ~ComplexPolygon() never called?

The base class Shape has a non-virtual destructor. When delete shapePtr is executed, the pointer type is Shape*, so deleting the derived object through this pointer without a virtual base destructor causes undefined behavior.

As a result, the derived destructor may not execute, and the dynamically allocated vertexArray memory may not be released.

2. State the C++ standard rule.

Deleting a derived-class object through a base-class pointer requires the base class to have a virtual destructor. Otherwise, the behavior is undefined.

This can cause memory leaks and improper resource cleanup.

3. How does the virtual destructor solve the problem?

Declaring the base destructor as virtual enables dynamic dispatch during deletion. The object's dynamic type is used to invoke the derived destructor first, followed by the base destructor.

The vtable and vptr mechanism supports runtime selection of the correct virtual destructor.

4. Corrected C++ program

```
#include <iostream>
using namespace std;

class Shape {
public:
   Shape() {
      cout << "Shape created." << endl;
   }

   virtual ~Shape() {
      cout << "Shape destroyed." << endl;
   }

   virtual void render() const {
      cout << "Rendering generic shape." << endl;
   }
};

class ComplexPolygon : public Shape {
private:
   double *vertexArray;
   int vertexCount;

public:
   ComplexPolygon(int count) : vertexCount(count) {
      vertexArray = new double[vertexCount * 2];
      cout << "Allocated " << vertexCount
           << " vertices on heap." << endl;
   }

   ~ComplexPolygon() {
      cout << "Deallocating polygon vertex heap memory."
           << endl;
      delete[] vertexArray;
   }

   void render() const override {
      cout << "Rendering polygon with "
           << vertexCount << " vertices." << endl;
   }
};

int main() {
   Shape *shapePtr = new ComplexPolygon(1000);

   shapePtr->render();

   delete shapePtr;

   return 0;
}
```

Expected Output

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
```

Shape destroyed.

Conclusion: The virtual destructor ensures proper destruction of derived objects and prevents memory leaks during polymorphic deletion.

QUESTION 2: Inverted Exception Catch Handlers and Unreachable Code

1. Identify the warning/error.

The compiler may generate a warning that the InsufficientFundsException handler will never be reached because the earlier BankingException handler already catches it.

2. Why does the base handler match the derived exception?

InsufficientFundsException inherits from BankingException.

Therefore, an object of the derived class can be caught through a reference to its base class.

The following handler can catch both base and derived exceptions:

catch (const BankingException &be)

3. State the rule for ordering exception handlers.

In C++, derived-class exception handlers must be placed before base-class handlers.

This ensures that specific exceptions are handled before general exceptions.

4. Corrected try-catch block

```
#include <iostream>
#include <string>
using namespace std;

class BankingException {
protected:
   string msg;

public:
   BankingException(string m) : msg(m) {}

   virtual string getMessage() const {
      return msg;
   }
};

class InsufficientFundsException : public BankingException {
private:
```

```
    double shortfall;

public:
    InsufficientFundsException(string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const {
        return shortfall;
    }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException(
            "Account overdrawn!", amount - balance);
    }
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException &ife) {
        cout << "Shortfall of: $"
             << ife.getShortfall() << endl;
    }
    catch (const BankingException &be) {
        cout << "General Banking Error: "
             << be.getMessage() << endl;
    }
    catch (...) {
        cout << "Unknown error occurred." << endl;
    }

    return 0;
}
```

Expected Output

```
Shortfall of: $250
```

Conclusion: Placing derived exception handlers before base handlers ensures that specific exceptions are handled correctly.

QUESTION 3: Exception Rethrowing Slicing Bug

1. Why is the TimeoutError handler bypassed?

The statement `throw e;` creates a new exception object using the static type of `e`, which is `DatabaseError`.

The derived information from `TimeoutError`, including `timeoutMs`, is lost. This is called **object slicing**.

Therefore, the TimeoutError handler is bypassed.

2. Difference between `throw e;` and `throw;`

- `throw e;` creates and throws a copy of the exception using the static type of the expression. This can cause object slicing.
- `throw;` rethrows the original active exception object without losing its derived type.

3. Correct the rethrow statement.

Replace:

```
throw e;
```

With:

```
throw;
```

4. Corrected C++ program

```
#include <iostream>
#include <exception>
#include <string>
using namespace std;

class DatabaseError : public exception {
protected:
   string errorDetails;

public:
   DatabaseError(string msg) : errorDetails(msg) {}

   const char* what() const noexcept override {
      return errorDetails.c_str();
   }
};

class TimeoutError : public DatabaseError {
public:
   int timeoutMs;

   TimeoutError(string msg, int ms)
      : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
   throw TimeoutError(
      "Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
   try {
      executeQuery();
   }
   catch (const DatabaseError &e) {
      cout << "[AUDIT LOG]: Intercepted DB error."
```

```
            << endl;
        throw;
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError &te) {
        cout << "Client caught TimeoutError! Duration: "
             << te.timeoutMs << " ms" << endl;
    }
    catch (const DatabaseError &de) {
        cout << "Client caught generic DatabaseError: "
             << de.what() << endl;
    }

    return 0;
}
```

Expected Output

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Conclusion: Using bare `throw;` preserves the original exception object and prevents object slicing.

QUESTION 4: Attempting to Instantiate an Incomplete Abstract Class

1. State the compilation error.

The compiler reports an error similar to:

```
Cannot declare variable 'packet' to be of abstract type 'VideoPacket'.
```

This occurs because `VideoPacket` has not implemented all the pure virtual functions inherited from `NetworkPacket`.

2. What makes a class abstract?

A class containing at least one pure virtual function is called an abstract class.

Example:

```
virtual void serialize() = 0;
```

An abstract class cannot be directly instantiated.

3. When does a derived class become concrete?

A derived class becomes concrete when it provides implementations for all inherited pure virtual functions.

If even one pure virtual function remains unimplemented, the derived class also remains abstract.

4. Corrected C++ program

```
#include <iostream>
using namespace std;

class NetworkPacket {
public:
   virtual ~NetworkPacket() {}

   virtual void serialize() = 0;
   virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
   int frameID;

public:
   VideoPacket(int id) : frameID(id) {}

   void serialize() override {
      cout << "Serializing H.264 Video Frame: "
           << frameID << endl;
   }

   int getPacketSize() override {
      return 1024;
   }
};

int main() {
   VideoPacket packet(1042);

   packet.serialize();

   cout << "Packet Size: "
        << packet.getPacketSize() << " bytes" << endl;

   return 0;
}
```

Expected Output

```
Serializing H.264 Video Frame: 1042
Packet Size: 1024 bytes
```

Conclusion: Implementing all pure virtual functions makes the derived class concrete and allows object creation.

QUESTION 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

1. Why does the program print the base-class message?

The function triggerEncounter(Entity e) accepts an object by value.

When a Dragon object is passed, only the base-class portion is copied into e. This is called object slicing.

Therefore, the derived-class information is lost, and Entity::attack() is executed.

2. Explain the role of vptr.

The vptr is a hidden pointer maintained by the compiler for objects with virtual functions.

- **Pass-by-value:** Creates a separate base-class object. The derived portion is sliced away, so the copied object uses the base-class virtual function table.
- **Pass-by-reference:** Does not create a sliced copy. The original object's dynamic type is preserved, allowing the correct overridden function to execute.

3. Correct the function.

Change:

```
void triggerEncounter(Entity e)
```

To:

```
void triggerEncounter(const Entity &e)
```

4. Corrected C++ program

```
#include <iostream>
using namespace std;

class Entity {
public:
   virtual ~Entity() {}

   virtual void attack() const {
      cout << "Entity performs generic scratch."
           << endl;
   }
};

class Dragon : public Entity {
public:
```

```
    void attack() const override {
        cout << "Dragon breathes scorching fire!"
             << endl;
    }
};

void triggerEncounter(const Entity &e) {
    e.attack();
}

int main() {
    Dragon boss;

    triggerEncounter(boss);

    return 0;
}
```

Expected Output

Dragon breathes scorching fire!

Conclusion: Passing objects by reference preserves runtime polymorphism and enables dynamic binding.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

QUESTION 1: Universal Payment Gateway with Runtime Polymorphism

1. Structured Pseudocode

Abstract Base Class: PaymentGateway

```
ABSTRACT CLASS PaymentGateway

    PURE VIRTUAL METHOD Authorize(amount AS REAL)
        RETURNS BOOLEAN

    PURE VIRTUAL METHOD Capture()
        RETURNS BOOLEAN

    VIRTUAL DESTRUCTOR

END CLASS
```

Derived Class: CreditCardGateway

```
CLASS CreditCardGateway INHERITS PaymentGateway

   METHOD Authorize(amount)
      READ cardNumber
      READ CVV

      IF cardNumber has 16 digits AND CVV has 3 digits
         PRINT "Card Authorized"
         RETURN TRUE
      ELSE
         PRINT "Invalid Card Details"
         RETURN FALSE
      END IF
   END METHOD

   METHOD Capture()
      PRINT "Credit Card Payment Captured"
      RETURN TRUE
   END METHOD

END CLASS
```

Derived Class: CryptoGateway

```
CLASS CryptoGateway INHERITS PaymentGateway

   METHOD Authorize(amount)
      READ walletHash
      READ gasFee

      IF walletHash starts with "0x"
         IF gasFee is valid
            PRINT "Crypto Payment Authorized"
            RETURN TRUE
         END IF
      END IF

      PRINT "Crypto Authorization Failed"
      RETURN FALSE
   END METHOD

   METHOD Capture()
      PRINT "Cryptocurrency Payment Captured"
      RETURN TRUE
   END METHOD

END CLASS
```

Main Program

```
START

   DECLARE PaymentGateway* gatewayPtr

   CREATE CryptoGateway object
   ASSIGN object to gatewayPtr

   IF gatewayPtr->Authorize(1500.00) = TRUE
      gatewayPtr->Capture()
```

```
    END IF

    DEALLOCATE gatewayPtr

STOP
```

2. Dry Run – Cryptocurrency Payment of $1,500

Step	Execution	Result
1	Create CryptoGateway object	Object created
2	Assign object to PaymentGateway pointer	Polymorphism enabled
3	Call Authorize(1500.00)	CryptoGateway method executes
4	Validate wallet prefix	Wallet must start with 0x
5	Validate gas fee	Gas fee checked
6	Call Capture()	Payment captured
7	Deallocate gatewayPtr	Virtual destructor executes

Expected Output:

```
Crypto Payment Authorized
Cryptocurrency Payment Captured
```

Conclusion: Runtime polymorphism allows a base-class pointer to execute the appropriate derived-class payment methods.

QUESTION 2: Resilient Banking Engine with Nested Try-Catch and Rethrowing

1. Structured Pseudocode

Exception Classes

```
CLASS TransactionException
    ATTRIBUTE reason AS STRING
END CLASS

CLASS AccountFrozenException
    INHERITS TransactionException
END CLASS

CLASS DailyLimitExceededException
    INHERITS TransactionException
    ATTRIBUTE excessAmount AS REAL
END CLASS
```

Low-Level Function: DebitLedger()

```
FUNCTION DebitLedger(amount)

   IF account is frozen
      THROW NEW AccountFrozenException(
         "Account is frozen")
   END IF

   IF amount exceeds daily limit of 50000
      THROW NEW DailyLimitExceededException(
         "Daily limit exceeded", amount - 50000)
   END IF

   PRINT "Amount Debited Successfully"

END FUNCTION
```

Intermediate Function: AuditTransactionWrapper()

```
FUNCTION AuditTransactionWrapper(amount)

   TRY
      CALL DebitLedger(amount)

   CATCH (e AS TransactionException)
      WRITE "Transaction error recorded in audit log"
      RETHROW
   END TRY

END FUNCTION
```

Main Program

```
START

   TRY
      CALL AuditTransactionWrapper(1000)

   CATCH (e AS AccountFrozenException)
      PRINT "Transaction Failed: Account is Frozen"

   CATCH (e AS DailyLimitExceededException)
      PRINT "Daily Limit Exceeded"

   CATCH (e AS TransactionException)
      PRINT "General Transaction Error"

   CATCH (...)
      PRINT "Unknown Error"

   END TRY

STOP
```

2. Dry Run – $1,000 Withdrawal on a Frozen Account

Step	Execution	Result
1	Main calls AuditTransactionWrapper(1000)	Control enters wrapper

2	Wrapper calls DebitLedger(1000)	Debit operation begins
3	System checks account status	Account is frozen
4	AccountFrozenException is thrown	Control transfers to wrapper catch
5	Audit log is written	Error recorded
6	RETHROW executes	Original exception propagates
7	Main catches AccountFrozenException	Specific handler executes
8	Client notice is displayed	Withdrawal rejected

Expected Output:

```
Transaction error recorded in audit log
Transaction Failed: Account is Frozen
```

Conclusion: Nested exception handling and rethrowing allow errors to be logged at an intermediate level and handled specifically at the main level.

QUESTION 3: Polymorphic Game Engine with Virtual Destructors

1. Structured Pseudocode

Abstract Base Class: GameObject

```
ABSTRACT CLASS GameObject

   PURE VIRTUAL METHOD Update(deltaTime AS REAL)

   PURE VIRTUAL METHOD Render()

   VIRTUAL DESTRUCTOR

END CLASS
```

Derived Class: ParticleSystem

```
CLASS ParticleSystem INHERITS GameObject

   ATTRIBUTE particleCoordinates AS ARRAY

   CONSTRUCTOR
      ALLOCATE particleCoordinates dynamically
      PRINT "Particle System Created"
   END CONSTRUCTOR

   METHOD Update(deltaTime)
      UPDATE particle positions
   END METHOD
```

```
    METHOD Render()
        PRINT "Rendering Particles"
    END METHOD

    DESTRUCTOR
        DEALLOCATE particleCoordinates
        PRINT "Particle Memory Released"
    END DESTRUCTOR

END CLASS
```

Driver Program

```
START

    DECLARE GameObject* sceneEntities[2]

    ALLOCATE ParticleSystem objects
    STORE objects in sceneEntities

    FOR each object in sceneEntities
        CALL Update(deltaTime)
        CALL Render()
    END FOR

    FOR each object in sceneEntities
        DEALLOCATE sceneEntities[i]
    END FOR

STOP
```

2. Cleanup Phase and Destructor Order

When a derived object is deleted through a base-class pointer, the virtual destructor ensures that the derived destructor executes first, followed by the base destructor.

Destructor Execution Order:

1. `ParticleSystem` destructor executes.
2. Dynamically allocated particle memory is released.
3. `GameObject` base destructor executes.
4. The object is completely destroyed.

Trace Table

Step	Operation	Result
1	DEALLOCATE sceneEntities[0]	Derived destructor called
2	Release particle coordinates	Heap memory freed
3	Execute base destructor	Base object destroyed
4	Repeat for remaining objects	All objects cleaned up

Conclusion: Virtual destructors ensure proper resource cleanup and prevent memory leaks in polymorphic game engines.

QUESTION 4: File Stream Pipeline with Stack Unwinding and Catch-All

1. Structured Pseudocode

Class: ResourceGuard

```
CLASS ResourceGuard

    ATTRIBUTE fileHandle

    CONSTRUCTOR
        ACQUIRE file handle
        PRINT "Resource Acquired"
    END CONSTRUCTOR

    DESTRUCTOR
        RELEASE file handle
        PRINT "Resource Released"
    END DESTRUCTOR

END CLASS
```

Custom Exception: CRCValidationException

```
CLASS CRCValidationException
    ATTRIBUTE expectedChecksum
    ATTRIBUTE calculatedChecksum
END CLASS
```

Function: IngestDataStream()

```
FUNCTION IngestDataStream(fileName)

    CREATE ResourceGuard guard

    OPEN fileName

    READ data blocks
    CALCULATE CRC checksum

    IF expectedChecksum != calculatedChecksum
        THROW NEW CRCValidationException
    END IF

    PRINT "File Validation Successful"

END FUNCTION
```

Main Program

```
START
```

```
    TRY
        CALL IngestDataStream("data.txt")

    CATCH (e AS CRCValidationException)
        PRINT "CRC Validation Failed"
        PRINT expectedChecksum
        PRINT calculatedChecksum

    CATCH (...)
        PRINT "Unknown Error Occurred"

    END TRY

STOP
```

2. Stack Unwinding Trace

When a CRC mismatch occurs, the exception is thrown. The local ResourceGuard object is automatically destroyed before the catch handler executes.

Trace Table

Step	Event	Result
1	Main calls IngestDataStream()	Function starts
2	ResourceGuard object created	File handle acquired
3	Data blocks are read	CRC calculated
4	CRC mismatch detected	Exception thrown
5	Stack unwinding begins	Local objects destroyed
6	ResourceGuard destructor executes	File handle released
7	Main catch handler executes	CRC error displayed

Expected Output:

```
Resource Acquired
Resource Released
CRC Validation Failed
```

Conclusion: Stack unwinding automatically destroys local objects during exception propagation. RAII ensures that resources are released safely.

QUESTION 5: Scientific Telemetry Calibration with noexcept Contracts

1. Structured Pseudocode

Custom Exception: OutOfBoundsException

```
CLASS OutOfBoundsException
   ATTRIBUTE reason AS STRING
END CLASS
```

Function: CalibrateThermalSensors()

```
FUNCTION CalibrateThermalSensors(rawVoltage)
   NOEXCEPT(FALSE)

   CALCULATE temperature from rawVoltage

   IF temperature < -273.15 OR temperature > 1500.0
      THROW NEW OutOfBoundsException(
         "Temperature outside valid range")
   END IF

   RETURN temperature

END FUNCTION
```

Function: CommitTelemetry()

```
FUNCTION CommitTelemetry(finalTemp) NOEXCEPT

   TRY
      SEND finalTemp to avionics hardware bus
      PRINT "Telemetry Committed"

   CATCH (...)
      PRINT "Hardware Error Suppressed"

   END TRY

END FUNCTION
```

Main Program

```
START

   TRY
      SET rawVoltage = faulty voltage

      SET finalTemp = CalibrateThermalSensors(rawVoltage)

      CALL CommitTelemetry(finalTemp)

   CATCH (e AS OutOfBoundsException)
      PRINT "Calibration Error: Temperature Out of Bounds"
```

```
  CATCH (...)
    PRINT "Unexpected Error"

  END TRY

STOP
```

2. Dry Run – Faulty Temperature Reading of -300°C

Step	Execution	Result
1	Main starts	Calibration begins
2	CalibrateThermalSensors() executes	Temperature calculated
3	Temperature is -300°C	Outside valid range
4	OutOfBoundsException is thrown	Control leaves function
5	Main catches exception	Specific handler executes
6	Error message displayed	Invalid reading rejected
7	CommitTelemetry() is skipped	No invalid data committed

Expected Output:

```
Calibration Error: Temperature Out of Bounds
```

Explanation:

The `noexcept` specifier indicates that `CommitTelemetry()` must not allow exceptions to escape. Its internal catch-all handler suppresses exceptions raised during the hardware operation.

The calibration function is allowed to throw an exception because it is declared as potentially throwing.

Conclusion: Exception specifications and `noexcept` contracts improve reliability by controlling error propagation in critical telemetry systems.

CONCLUSION

Virtual functions and runtime polymorphism allow C++ programs to execute derived-class methods through base-class pointers and references. Virtual destructors ensure proper memory cleanup. Exception handling provides structured error management through `try`, `catch`, and `throw`. Stack unwinding and RAII help release resources safely, while `noexcept`provides guarantees for functions that must not allow exceptions to escape.

Thus, these concepts improve code reusability, reliability, resource management, and runtime flexibility in C++ applications.